Advanced Database Systems — Practical Portfolio

Case study: **ShopStack**, a fictional multi-region online retailer (UK, EU, US)

Ewetola Mojetioluwa · 301897504 · MIT 8103 Advanced Database Systems · 2026/2027, First Semester · 27 October 2026

> Source repository (private): https://github.com/Mojetioluwa03/advanced-database-portfolio

1. What the portfolio contains

One case study runs through five activities, so each one starts from the database the previous one left behind. The dataset is deterministic: `seed.py` with a fixed random seed produces the same 26-table, 1.35-million-row ShopStack every time, which is why any plan or timing quoted below can be regenerated. Every claim is backed by output captured next to the script that produced it; database timings are server-side, warm-up discarded, median of three runs.

Activity	What was built	Headline result
1. Design 7 marks	26-table relational schema with keys, referential actions, check constraints and a reconciliation view; Chen and crow's-foot ER diagrams; deterministic generator, bulk loader and post-load checks.	1.35M rows loaded in 3m44s; all integrity checks pass; normalised to 3NF and BCNF, with four deliberate denormalisations argued and audited by a reconciliation view.
2. Optimisation 7 marks	Eight business queries spanning date ranges, lateral lookups, anti-joins, ledger scans and reporting joins; six indexes; before/after `EXPLAIN (ANALYZE, BUFFERS)`.	Latest-payment query **147,910 → 27 ms** (5,455×); daily trading 21.7 → 3.5 ms. Two indexes were measured and deliberately rejected.
3. Transactions 7 marks	Commit, rollback and savepoint demonstrations; four two-session races covering lost update, isolation levels, deadlock and write skew.	Oversell reproduced — two paid orders from one unit, **with no constraint error**; two fixes measured. Write skew survives REPEATABLE READ and is stopped by SERIALIZABLE.
4. NoSQL 7 marks	10,000-product document model in MongoDB with the embed/reference boundary justified; five MQL queries, five SQL equivalents returning the same rows; three indexes; three experiments.	Product page assembly **14.3 → 3 ms** in the document store; the brand league table is **4× slower** there (22 vs 95 ms) because every array must be unwound.
5. Distributed 6 marks	Three jurisdiction shards, a physical streaming standby, a `postgres_fdw` router with fan-out views and routing functions; seven experiments including node-loss and a two-phase-commit probe.	Point lookup 0.66 ms routed / 1.95 ms fanned out; `count(*)` 196.9 ms through the view but **10.1 ms** with per-shard partials; 63.3% of stock movements reference an order on another shard.

2. Selected findings

2.1 An index is a hypothesis, and two of them failed (Portfolio 2)

Six indexes were created, and two were rejected on evidence rather than kept for completeness. The anti-join for "products never ordered" did not change plan and stayed at ~80 ms, and the carrier-delivery query kept its sequential scan because it returns 30% of the table and the planner correctly prefers a heap scan. Reporting those as non-results is more useful than claiming eight improvements: an index that the planner refuses is a fact about the data distribution, not a mistake to hide.

2.2 The constraint that never fired (Portfolio 3)

Two sessions each read one unit of stock, slept through a payment call, and each wrote back zero. The result was two paid orders, two sale movements, and one unit ever existed — and `CHECK (quantity >= 0)` never fired, because neither session asked the database to do arithmetic; both computed the new value in the application from a stale read. A constraint protects against a wrong value being written, not against a decision made on stale data. A row lock held across the payment call fixes it at the cost of serialising every checkout of that SKU; a guarded `UPDATE … WHERE quantity >= 1` fixes it without the wait. The same portfolio shows write skew surviving REPEATABLE READ, the SERIALIZABLE retry loop, and a deadlock whose victim's first update rolls back cleanly.

2.3 The document model won the page and lost the league table (Portfolio 4)

Embedding brand, categories and variants in one product document made page assembly 4.8× faster, because the page is fetched together and the relational version needs four joins. The same embedding made the cross-catalogue brand aggregation four times slower, because it must unwind every variant array that the page reads never wanted. The model is a good aggregate boundary for fetching a page and a bad one for counting across all pages; neither result generalises without the other.

2.4 Federation strategy matters more than sharding (Portfolio 5)

Three shards keyed by jurisdiction answer a point read in 0.66 ms; the same read through a `UNION ALL` view costs 1.95 ms because it visits all three. The larger effect is on aggregates: `count(*)` through the view takes 196.9 ms against the single node's 15.4 ms, because the planner cannot push the aggregate through the union and ships 200,000 rows. Asking each shard for its partial and combining three numbers in the router costs 10.1 ms. The point is not that distribution is faster; it is that the wrong federation strategy is an order of magnitude worse than the right one, and the plan — not intuition — shows which is which.

3. The distributed design, and its consistency argument (Portfolio 5)

Two domains have different owners, so they are partitioned separately. Orders and customers follow the jurisdiction of the customer's default address, which keeps a checkout, its payment and its shipment in one database and therefore inside one ACID transaction. Stock follows the warehouse, so the ledger write that must never be lost is local to the node that performed the movement. The catalogue is replicated whole to every shard because every region reads it and nobody writes it at checkout pace; the sale snapshots `unit_price` into the order line, so a stale description cannot change what a customer is charged.

The consistency posture is chosen per domain rather than for the system. Stock and money are CP: if the warehouse shard is unreachable the sale is refused, because an oversell costs a refund and goodwill while a deferred sale costs nothing. Catalogue reads and reports are EL: a replica may serve a page that is hundreds of milliseconds old, measured at 18 KB / 465 ms of lag with a 338 ms catch-up. The ledger is durable and append-only, weak only where it points at an order on another shard — 63.3% of sale movements do — which is why the foreign key is replaced by a router-level reconciliation query that finds zero orphans.

Checkout is the one operation spanning two shards, and the stack cannot make it atomic: `postgres_fdw` refuses `PREPARE TRANSACTION` once a foreign table has been touched. The documented design is therefore a saga — order first, stock second, idempotent retry, scheduled reconciliation — ordered so that the recoverable failure is the one that can happen. It is analysed and costed rather than implemented, and the documentation says so.

4. Reflection

Measurement changed the conclusions, repeatedly. I predicted the document store would win the brand aggregation and it lost by 4×; I predicted federation would cost parity and the pushed-down aggregate beat the single node; I assumed a check constraint would stop an oversell and it never fired. Each correction is kept in the repository as an experiment or a negative result rather than quietly edited out, because the wrong prediction is evidence about how the system actually behaves.

The constraints shaped the engineering, and are stated rather than hidden. There was no Docker, no administrative rights and no cloud account, so PostgreSQL and MongoDB ran from portable archives and the Portfolio 5 cluster is five `pg_ctl`-managed processes rather than containers; that deviation from the planned "containerised configuration" is declared in the P5 README. The "network" between shards is loopback, which understates WAN latency, so the evidence is the ratios — one shard versus three, shipped rows versus pushed-down aggregates — not the absolute milliseconds.

The judgement call I am least certain of is replacing the movement-to-order foreign key with reconciliation. The alternative — keying movements by the customer so the reference stays local — would make every warehouse query fan out, moving the cost from a periodic check to every pick. I chose the periodic check, and it is defensible only because that check is cheap; if it were expensive, the shard key would have to change. The other open item is operational rather than technical: no automated failover, so a shard loss needs an operator.

What I would do next with more time: give every shard its own standby behind a failover manager, replace the catalogue rebuild with logical replication from a single catalogue database, and build the checkout saga end to end with a request-scoped idempotency key and a compensator, so the cross-shard path is demonstrated rather than analysed.

5. AI use declaration

Item	Detail
Tool	Qoder, an AI coding agent (Claude models). An earlier scaffold draft was produced with Claude in a first repository; it was reviewed against the brief and superseded by this implementation.
Used for	Drafting schema and DDL, the deterministic seed generator, query and index candidates, experiment harnesses (two-session races, replication and node-loss scripts), and drafting this documentation and the READMEs. No query result or timing was fabricated: every number quoted was produced by executing the committed scripts.
Review performed	Every script was executed against PostgreSQL 16.15 or MongoDB 9.0.4 on this machine; captured output is committed beside each script. Plans and timings were read and compared against expectations; claims contradicted by measurement were rewritten (the P4 index rule, the P5 aggregate assumption, the P3 constraint assumption). AI-drafted text was edited to match what the results actually show, and every number quoted here is traceable to a file in the repository.

6. Evidence and reproduction

Each activity folder contains a README stating what was done, how it was tested, and which files hold the captured output. The root README gives five reproduction routes; the essential sequence is: run `seed.py`, load the schema and data, then run the activity's own scripts, which rewrite the evidence they capture. Portfolio 4 needs MongoDB on port 27018; Portfolio 5 needs the Portfolio 1 cluster on 5433 and starts its shards on 5441–5443 with a standby on 5444. The dataset is regenerable, so any plan or timing in this document can be reproduced, and where a result was negative it is reported as such.

I declare that the database implementation, analysis, documentation and conclusions submitted here are my own work, that AI assistance is declared above in line with the University's academic integrity requirements, and that I can explain and defend every part of the submitted work.

Ewetola Mojetioluwa · 301897504 · 27 October 2026